

ColorRoom

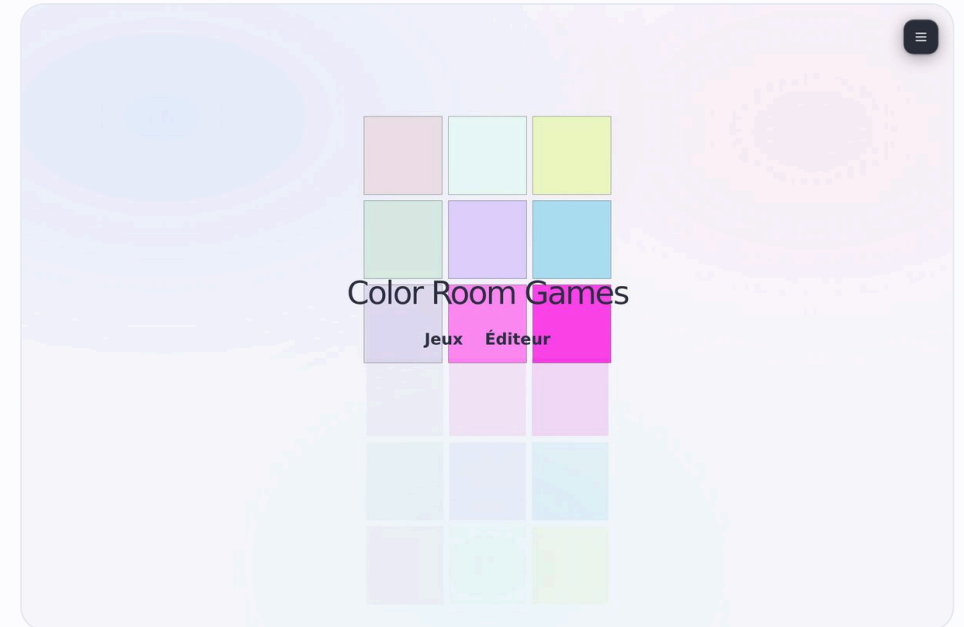
Serious games pédagogiques sur les plaques lumineuses de la ColorRoom

Téo Trompier · candidat E2 · sous-équipe JavaScript

Lycée Édouard Branly · Lyon

LUMEN – Cité de la Lumière · **ENTPE / LTDS** · labo BPMNP

PILE TECHNIQUE





Sommaire



01 Contexte et commanditaire



03 Besoin et cas d'utilisation



05 Architecture et conception



07 Réseau et déploiement



09 Sécurité et comptes



11 Mesure et chromaticité



02 Le système ColorRoom



04 Équipe et planification



06 Choix techniques et pile



08 Interface et base de données



10 Jeux, IA et multijoueur



12 Qualité, bilan et démo



Contexte et partenaire

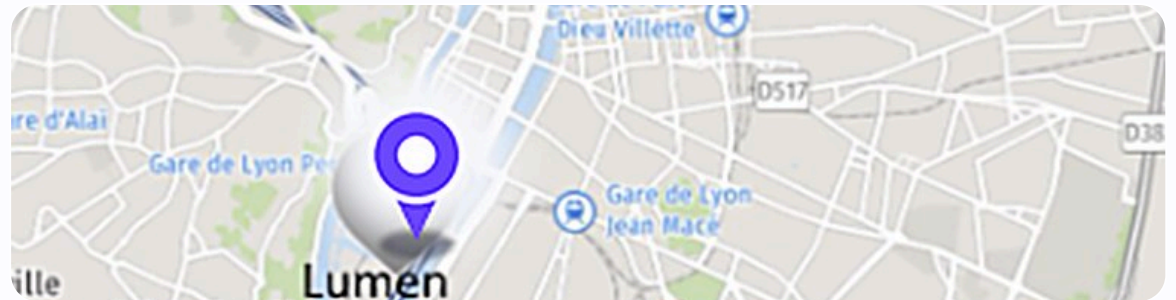
- Commanditaire : laboratoire de recherche **ENTPE / LTDS**, équipe **BPMNP**
- ColorRoom hébergée à **LUMEN – La Cité de la Lumière** (Lyon Confluence)
- Équipement scientifique : **2 cellules jumelles** d'analyse des effets colorés
 - Contacts : M. Labayrade, M. Vella · Professeur : M. Delbosc

ENTPE (École Nationale des Travaux Publics de l'État) · **LTDS** (Lab. de Tribologie et Dynamique des Systèmes)

BPMNP (Bio-ingénierie, Perception, Mécanique Numérique) · **LUMEN** (Cité de la Lumière)



 LUMEN · Cité de la Lumière (Lyon Confluence)



 Implantation : LUMEN (Confluence) & ENTPE (agglomération lyonnaise)



Deux cellules jumelles à visualiser

- **2 cellules jumelles** (2 salles d'analyse identiques)
- Chaque cellule est tapissée de plaques sur les **murs** ET le **plafond**
- **42 plaques** au total (**21 par cellule**) pilotées indépendamment
- Permet de recréer des **ambiances lumineuses** à spectres précis
 - Photo réelle : panneaux allumés sur murs et plafond



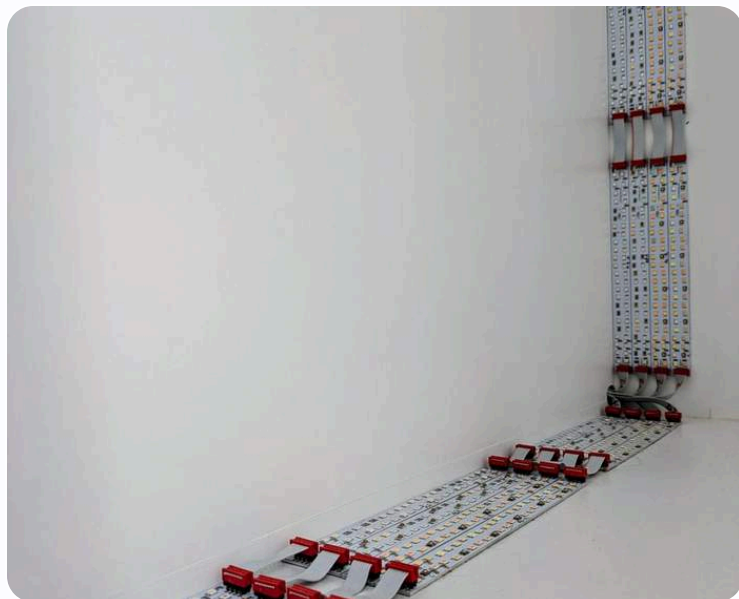
© Laboratoire de la couleur · Campus Lumière





LA PLAQUE LUMINEUSE

Dimensions et caractéristiques



LED en bandes sur les bords ·
dimensions

80 × 80
cm

, épaisseur **23**
~ cm



42

plaques (21 par cellule)



32

canaux = 24 spectres étroits + 8 blancs



2360

LED par plaque (~300 W max)



80×80

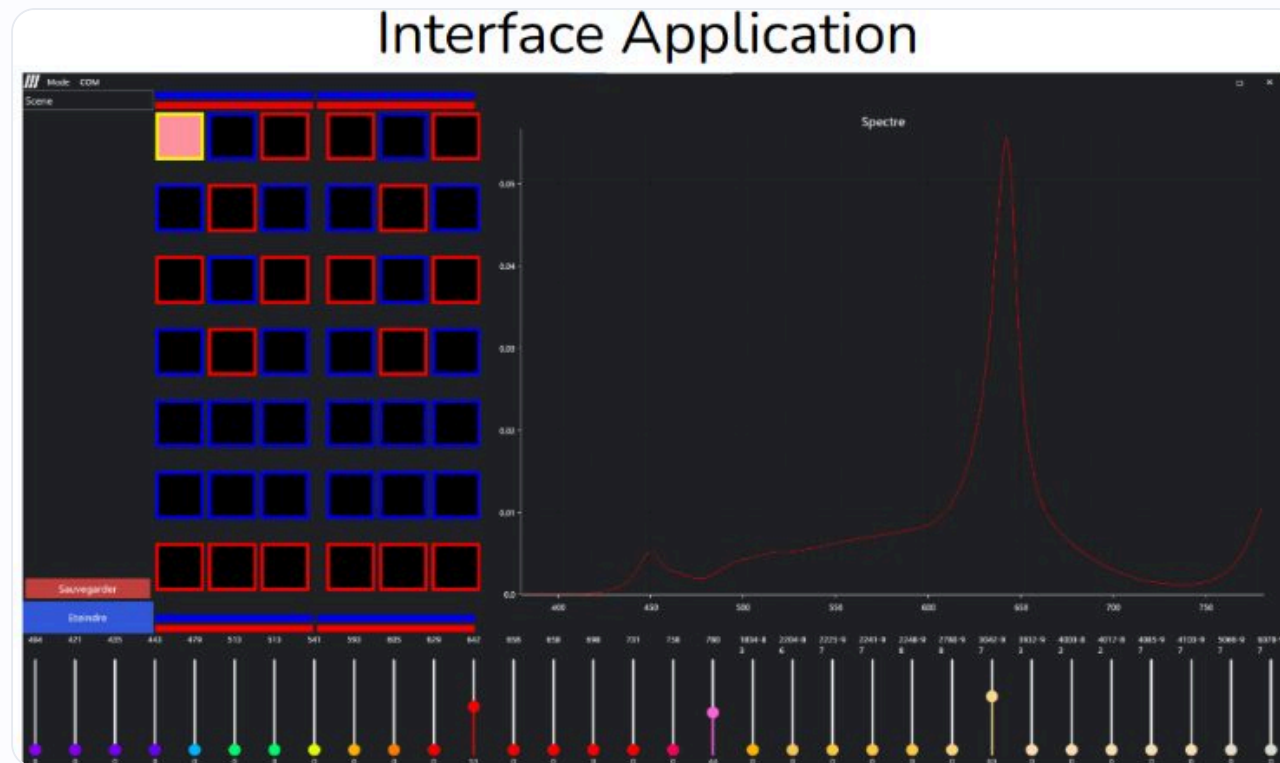
cm par plaque (ép. ~23 cm)

Pilotage en **RS-485** · spectre du **proche UV au proche IR** ; chaque plaque produit couleurs et intensités variées.



Pourquoi un nouveau logiciel ?

- Logiciel actuel **réservé aux chercheurs** de l'ENTPE
- Pilotage brut : grille des plaques, **32 sliders** de canaux, courbe de **spectre**
- **Trop complexe** pour une initiation : aucune dimension pédagogique
 - D'où ColorRoomGames : une couche **simple et ludique** par-dessus le matériel





Problématique et objectifs

Le logiciel de recherche de la ColorRoom est trop complexe pour l'initiation. Comment rendre la salle accessible aux apprenants, de façon ludique et pédagogique ?

🎯 Objectifs



Une série de **serious games** sur la lumière et la couleur



Accessible depuis un **navigateur**, sans installation



Adaptable à une **seule plaque** lumineuse

🔒 Contraintes



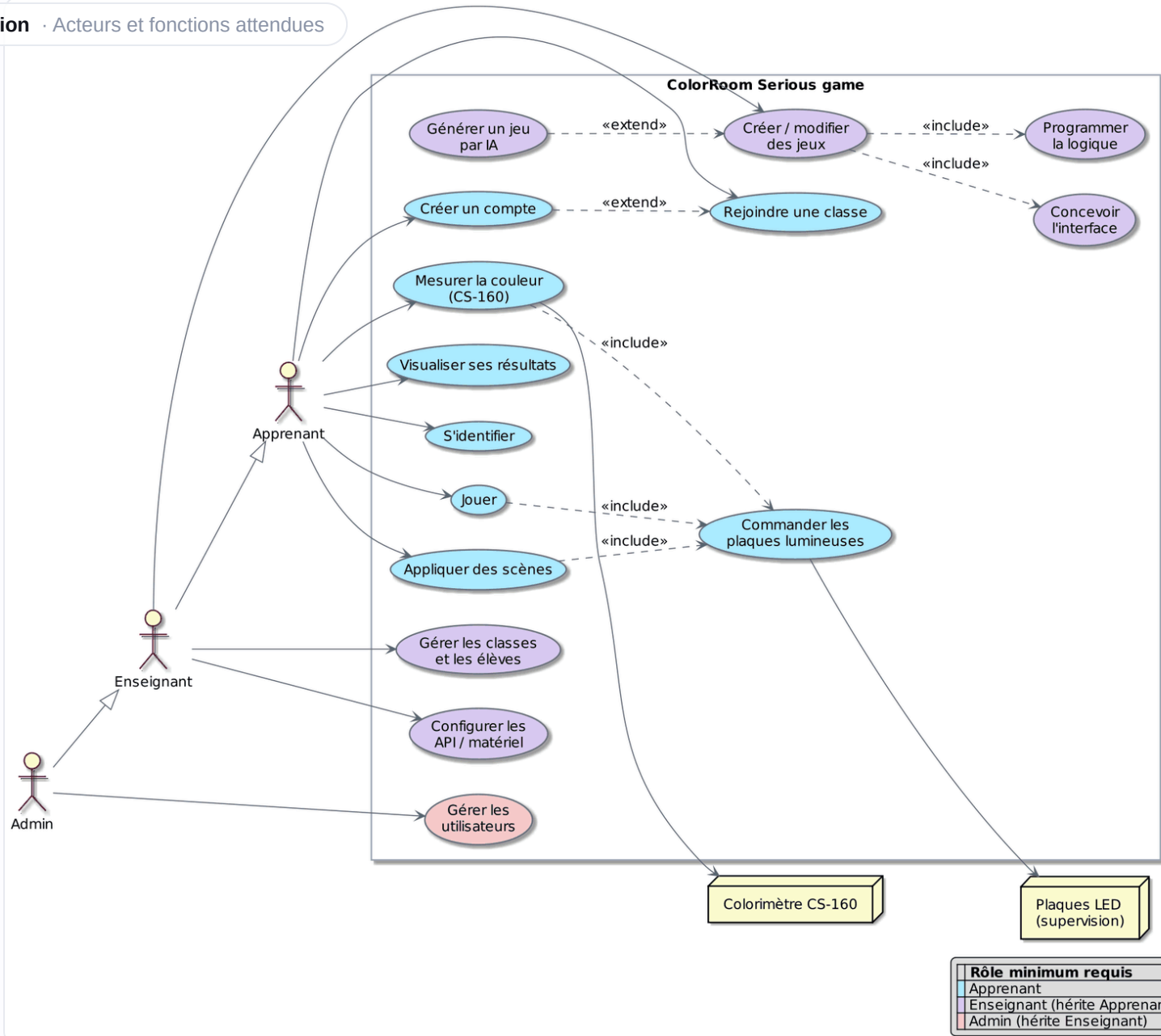
Raspberry Pi 5 + SSD, Docker imposé



Fonctionnement **hors-ligne** (réseau local)



Latence des plaques à compenser





Équipe et répartition



Équipe JavaScript · React

E1 → E4 · dont moi (E2)



Équipe Python · NiceGUI

E5 → E8

MB

E1 · Maxime Bonnevey

Infrastructure Pi & Docker, CI/CD, intégration colorimètre, sécurité



TT

E2 · Téo Trompier

MOI

Base de données, API, UI/UX, jeux solo + multijoueur, proxy LED, documentation



IA

E3 · Ilyes Arbadji

Éditeur de jeux no-code (hors de mon périmètre)

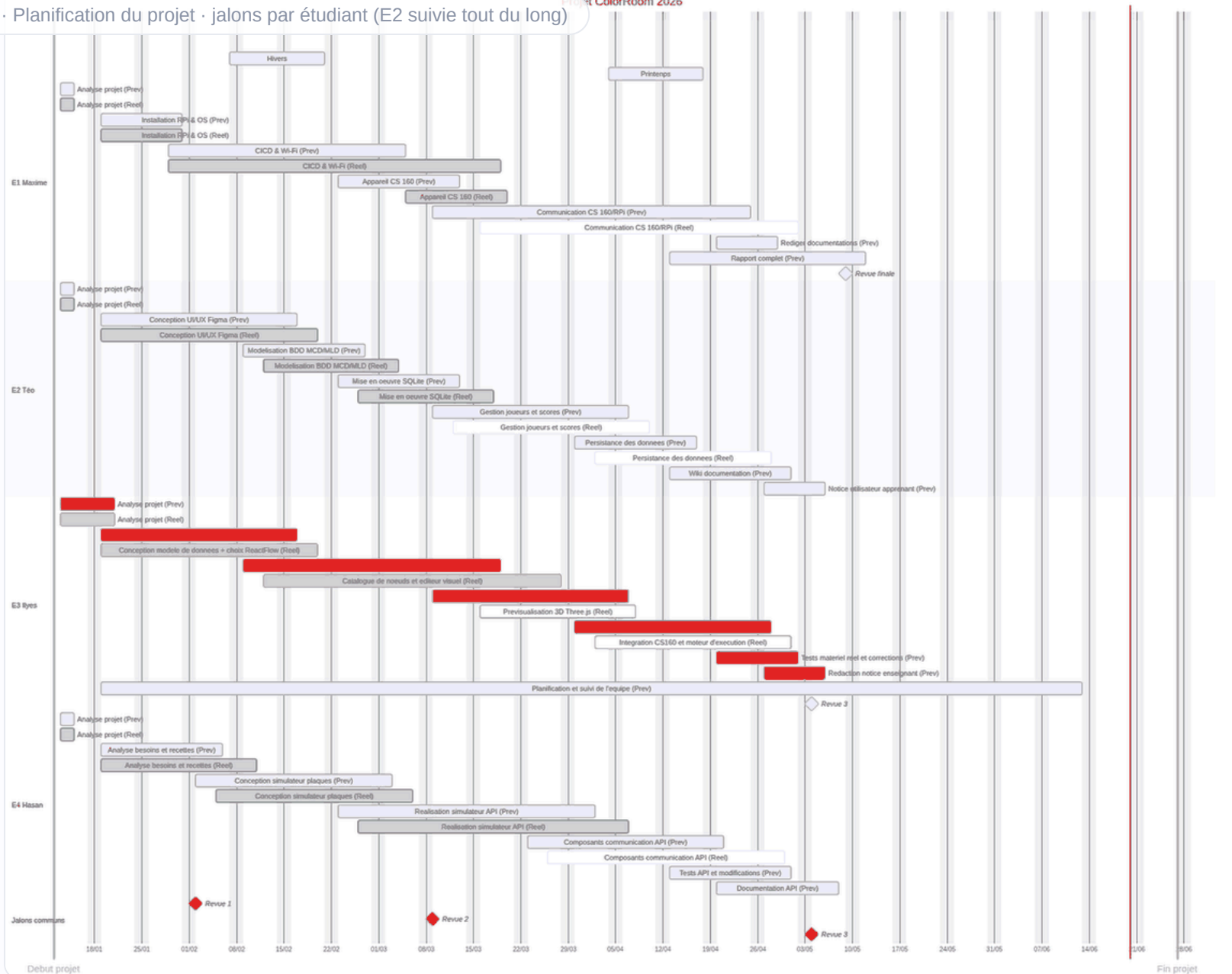


HA

E4 · Hasan Akyuz

Tests de l'API, simulateur de plaques, Swagger / Postman







Agile, suivi client et versionnement

🎯 Méthode & suivi



Démarche **agile** : développement **incrémental**, livraisons régulières, ajustements



Suivi des tâches via un **tableau Kanban** (À faire / En cours / Terminé)



Échanges réguliers avec le **client M. Labayrade** (directeur du labo **BPMNP**)



Documentation du projet rédigée par moi (guide technique, 15 diagrammes UML, notice)

</> Versionnement



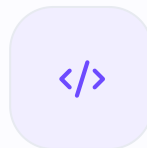
Git / GitHub : commits clairs (~280), historique lisible



Travail sur la branche **ux-last** (intégration) puis **merge sur main** (stable)



Chaîne **CI/CD** GitLab → build Docker → déploiement sur le Pi



PARTIE 2 · DE LA PRÉSENTATION GÉNÉRALE À MA CONTRIBUTION

Ma contribution · E2

Architecture & choix techniques · base de données · sécurité · interface 3D · jeux solo, IA et multijoueur ·
mesure colorimétrique





Architecture logicielle

- Serveur unique **Next.js** (App Router) conteneurisé
- **Route Handlers** + couche **lib/** (auth, db, services)
- Proxy HTTP de **supervision** ; pont **.NET** du CS-160
 - SQLite (better-sqlite3) · 100 % hors-ligne
 - Diagramme **logiciel** : le matériel (Raspberry Pi) figure sur le diagramme de **déploiement**

ColorRoom - Architecture

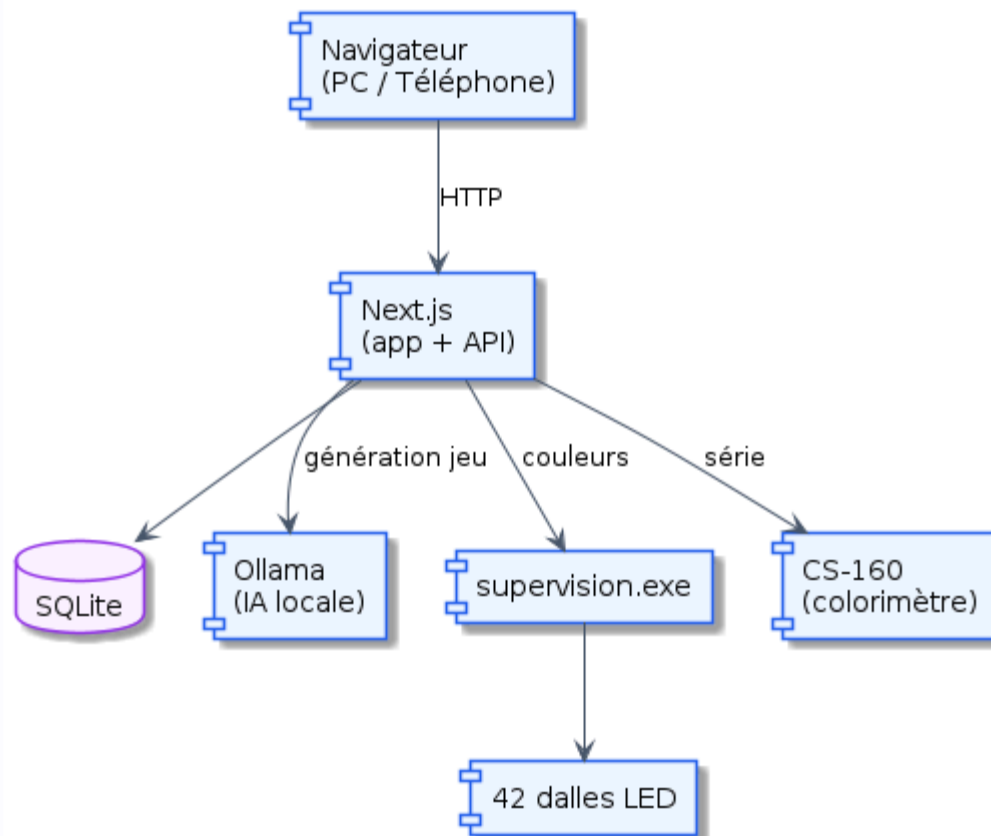
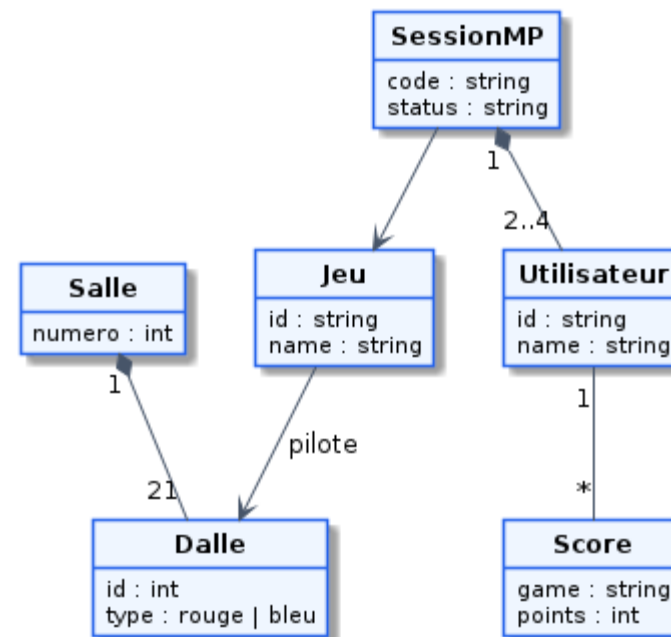




Diagramme de classes

- Entités métier modélisées et **typées en TypeScript**
- Utilisateur, Classe, Jeu, Score, Session...
- Relations **1-N** (une classe regroupe plusieurs apprenants)
- Vue **objet** du domaine, complémentaire du modèle relationnel

ColorRoom - Classes principales





Choix techniques · étude comparative

CRITÈRE

**JS + Node-RED**

PISTE ÉCARTÉE

**Next.js + React + TS**

RETENU

Paradigme

Programmation **par flux** (dataflow) : peu adapté à une appli multi-pages**Composants déclaratifs** (JSX/TSX) + routage **App Router**

Rendu / UI

Pas de composants ni de **Virtual DOM****Virtual DOM + réconciliation** : mise à jour minimale du DOM, état réactif

Typage

**JavaScript** dynamique : erreurs au **runtime****TypeScript** : typage statique, erreurs à la **compilation** (tsc + ESLint)

Maintenabilité

Flux **JSON** peu lisibles en revue de code / GitCode **modulaire** et **versionnable** (diffs Git, revue de code)

Back-end

Logique couplée au **moteur de flux****Route Handlers + Server Components** : un seul **runtime** Node (SSR)



La pile technique



Next.js

16.2 · App Router



React

19 · composants



TypeScript

5.5 · strict



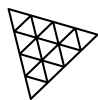
Node.js

runtime serveur



SQLite

better-sqlite3 11.5



Three.js

0.160 · vue 3D



Docker

multi-stage arm64



Raspberry Pi

5 · cible



CSS

langage de style



Lucide

icônes



Ollama

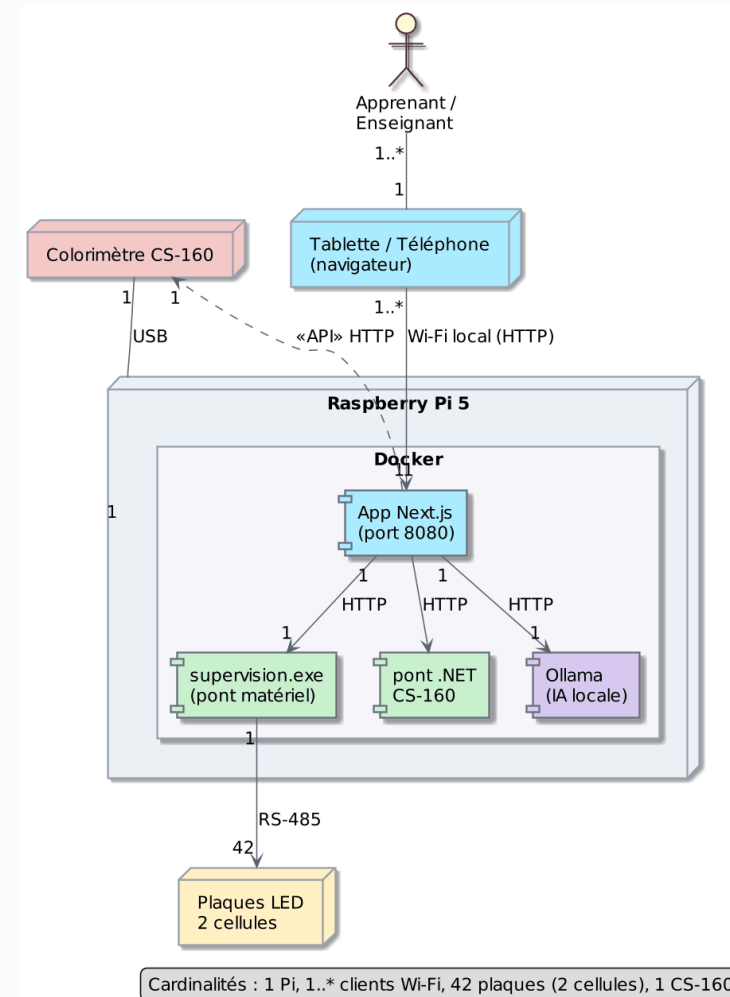
exécution de modèles IA en local

Tests automatisés avec
Playwright



Réseau et déploiement

- **Raspberry Pi 5 + Docker** (image arm64)
- **Dalles** : bus RS-485
- **CS-160** : USB + API
- **Clients** : **Wi-Fi local** (ColorRoom_WiFi)
- **Accès** : <http://172.17.40.39:8080>





Adresses des API et canaux LED

- Réglage des **URL d'API à chaud** (Supervision, CS-160) sans redémarrer le serveur
- Stockées en base ; repli sur les **variables d'environnement**
- Bouton **Tester (/health)** pour vérifier la liaison matériel
- Banc de test des **32 canaux LED** (0-255) par dalle, debounce 200 ms

The screenshot shows a web interface for configuring LED channels and API addresses. It includes sections for 'Configuration' (API addresses), 'API Supervision (dalles LED)', 'API CS-160 (colorimètre Konica)', and 'Contrôle des canaux LED (32)'. The 'Contrôle des canaux LED' section features a dropdown for 'Dalle' (set to 'Toutes (1-42)'), buttons for 'Tout au max', 'Éteindre', and 'Envoyer', and a grid of 32 color channels (Ch.0 to Ch.8) with sliders for intensity (0-255).

Configuration

Adresses des APIs (modifiables à chaud, sans redémarrer le serveur)

Indiquez l'URL complète **avec le port** (ex. `http://172.17.50.136:18080`). Laissez un champ vide pour revenir à la valeur par défaut / variable d'environnement.

API Supervision (dalles LED)

source : défaut
Pilote les 42 dalles. Test : GET /state

`http://172.17.50.136:18080`

API CS-160 (colorimètre Konica)

source : défaut
Bridge de mesure spectrale. Test : GET /api/health

`http://172.17.50.39:3000`

Enregistrer **Tester (/health)**

Contrôle des canaux LED (32)

Dalle : Toutes (1-42) **Tout au max** **Éteindre** **Envoyer**

Contrôlez chaque canal spectral (0-255). Les sliders envoient automatiquement (200 ms de debounce) sur la dalle sélectionnée.

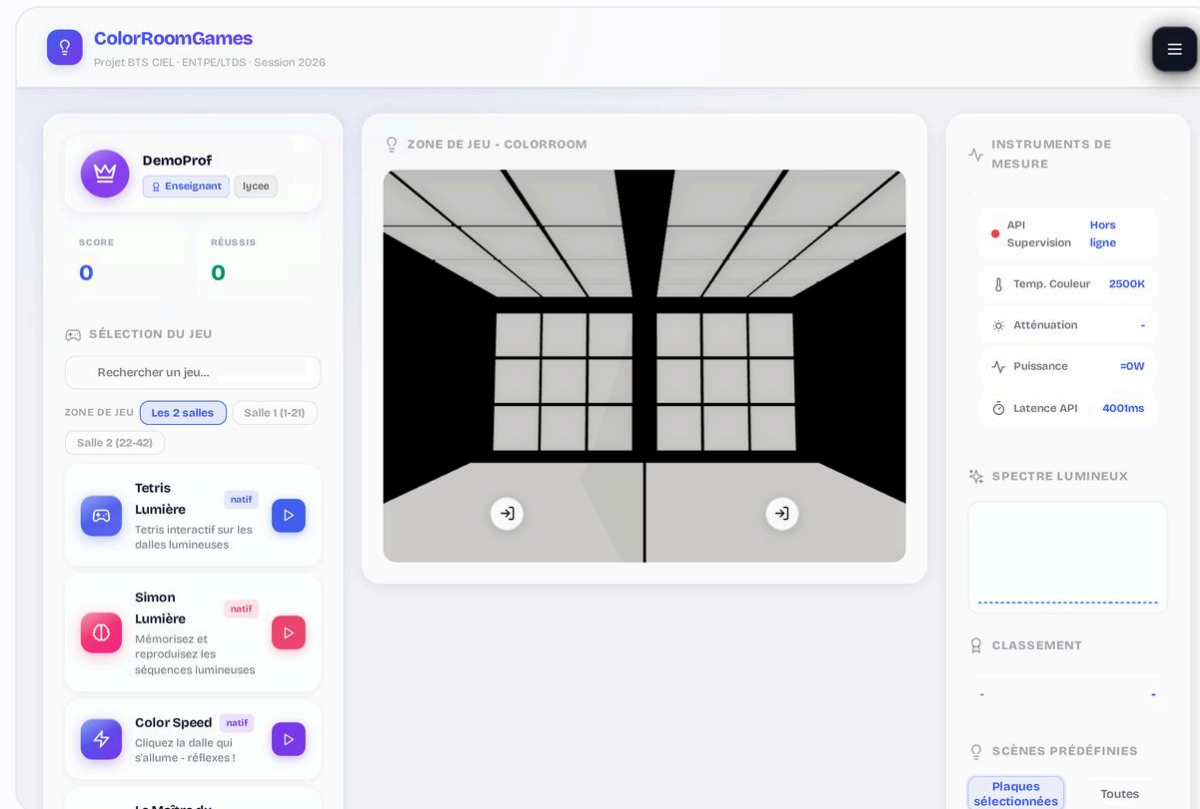
Type de dalle : **Rouge** Bleu

Ch.0 Violet 404nm	0	Ch.1 Violet clair 421nm	0	Ch.2 Bleu-violet 435nm	0
Ch.3 Bleu 443nm	0	Ch.4 Cyan-bleu 479nm	0	Ch.5 Vert 513nm	0
Ch.6 Vert (2) 513nm	0	Ch.7 Jaune-vert 541nm	0	Ch.8 Orange 593nm	0



Interface et design system

- Design system « **verre dépoli** » : variables CSS partagées
- Pages : accueil 3D, /jeux, /mesure, /chromaticite, /gestion, /aide
- **Menu dynamique** selon le rôle
- **Vue 3D** (Three.js : WebGLRenderer, Room3D, 2 cellules)
 - forceContextLoss au démontage · pause via Page Visibility API





Vue 3D de la salle (Three.js)

- **WebGLRenderer** : moteur qui transforme la scène 3D en image via le **GPU** (carte graphique)
- Une dalle = un **Mesh** à **géométrie BoxGeometry** + matériau **émissif** (le panneau « émet » sa couleur)
- Couleur mise à jour en direct :
`material.emissive.set(color)`
- Boucle `requestAnimationFrame` ; au démontage `forceContextLoss` (anti-fuite WebGL)

</> `app/app/_components/Room3D.tsx`
github.com/NewGenesisAgency/color_room/blob/main/app/app/_components/Room3D.tsx#L255-L545

```
app/app/_components/Room3D.tsx

// WebGLRenderer : rend la scène 3D via le GPU
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setPixelRatio(Math.min(devicePixelRatio, 1.5));
renderer.outputColorSpace = THREE.SRGBColorSpace; // couleurs fidèles
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(CAM_FOV, W/H, 0.05, 80);

// une dalle = panneau 3D (Box) au matériau émissif
const mat = new THREE.MeshStandardMaterial({ emissive: 0x000000 });
const plate = new THREE.Mesh(new THREE.BoxGeometry(PS, PS, PT), mat);
mat.emissive.set(color); scene.add(plate); // la dalle s'allume

function loop() { renderer.render(scene, camera); raf = requestAnimationFrame(loop); }
loop();
return () => { cancelAnimationFrame(raf); renderer.forceContextLoss(); };
```



Base de données SQLite

- **better-sqlite3** · requêtes **préparées** (anti-injection)
- Tables **créées automatiquement** au démarrage : on peut **relancer sans rien casser**
- **WAL** : on peut **lire pendant qu'on écrit** (pas de blocage)
- Si la base est occupée, l'app **réessaie** quelques secondes au lieu d'échouer
- Suppression d'un compte = ses données liées **supprimées en cascade**

crg_users comptes : pseudo, mot de passe haché, rôle

crg_classes classes (code + QR), créées par un enseignant

crg_class_members lien élève ↔ classe (users ↔ classes)

crg_sessions jetons de connexion → users

crg_scores scores par jeu → users

crg_games jeux (natifs + créés dans l'éditeur)

crg_mp_sessions / mp_players parties multijoueur (players → session)

crg_app_config réglages (URL des API)



Gestion de l'état



Persistante

Stockée durablement en SQLite (survit aux redémarrages, volume Docker). `lib/db`



Transactionnelle · atomique

`db.transaction()` : ACID, tout-ou-rien (user + classe).
`register`



Volatile (en mémoire)

`useRef` : état runtime des dalles/jeux, perdu au rechargement. `app/jeux`



Environnement

`process.env` : secrets (clé, mot de passe admin) via `.env`, hors Git.



Réactive

`useState` : déclenche le re-rendu de l'interface à chaque changement.



Concurrente

Sémaphore `HW_CONCURRENCY=2` : sérialise les accès au matériel. `batch`

Transactionnelle

```
// tout-ou-rien (ACID)
const tx = db.transaction() => {
  insertUser.run(...); // + adhésion classe
};
tx(); // BEGIN ... COMMIT / ROLLBACK
```

Atomique

```
let hwInFlight = 0; // compteur
if (hwInFlight < 2) // 2 slots max
  hwInFlight++;    // accès matériel
else await waitQueue(); // sinon : file
```

Volatile

```
const [score, setScore] = useState(0); // réactif
const comboRef = useRef(0); // pur, sans re-render
comboRef.current++; // mutation directe
```



Authentication (PBKDF2 + cookie)

- L'apprenant saisit identifiant + mot de passe
- L'API recalcule le hash via **verifyPassword** (PBKDF2)
- Si valide, création d'une **session** (token aléatoire)
- Renvoi d'un cookie **HttpOnly + SameSite** (30 j glissants)

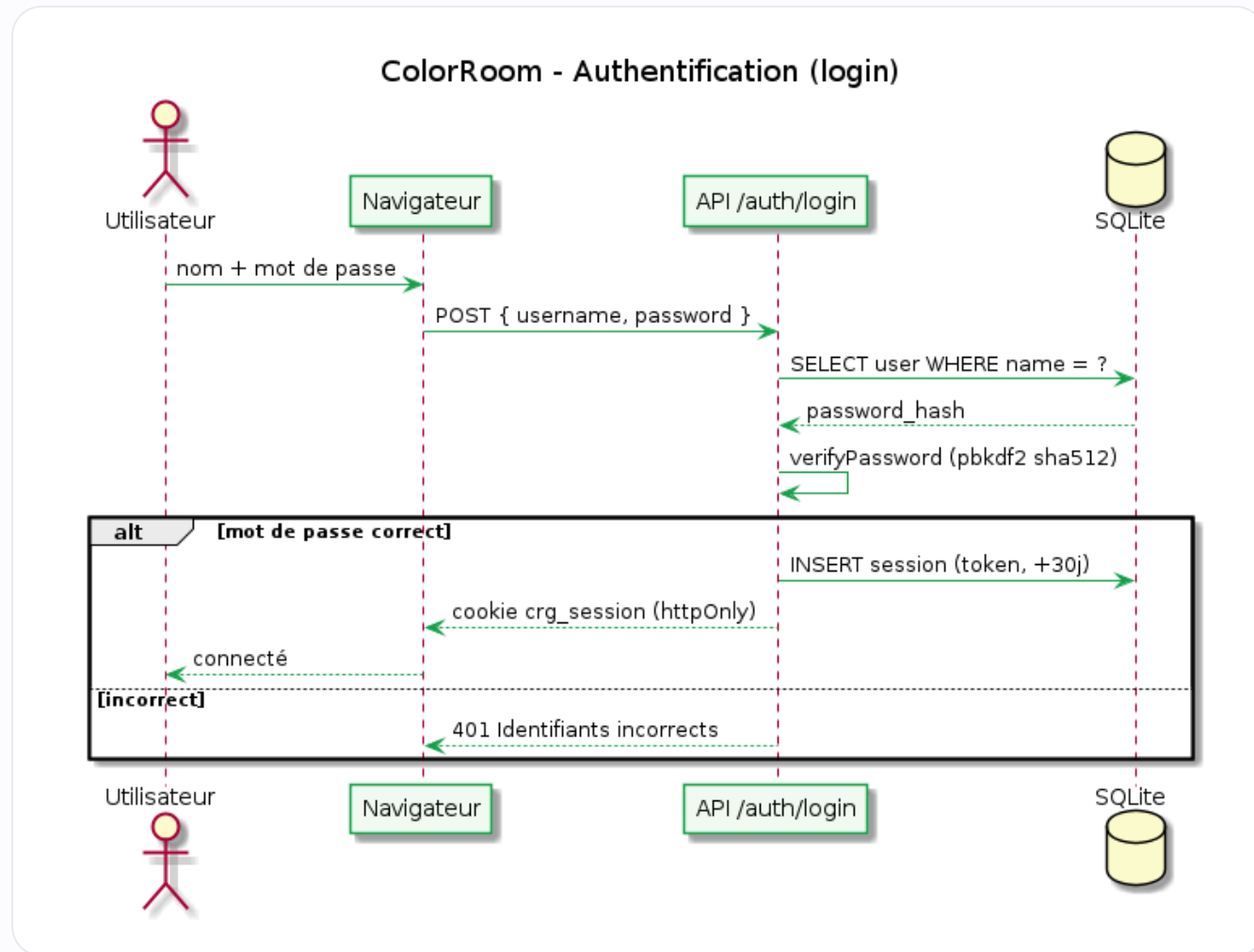
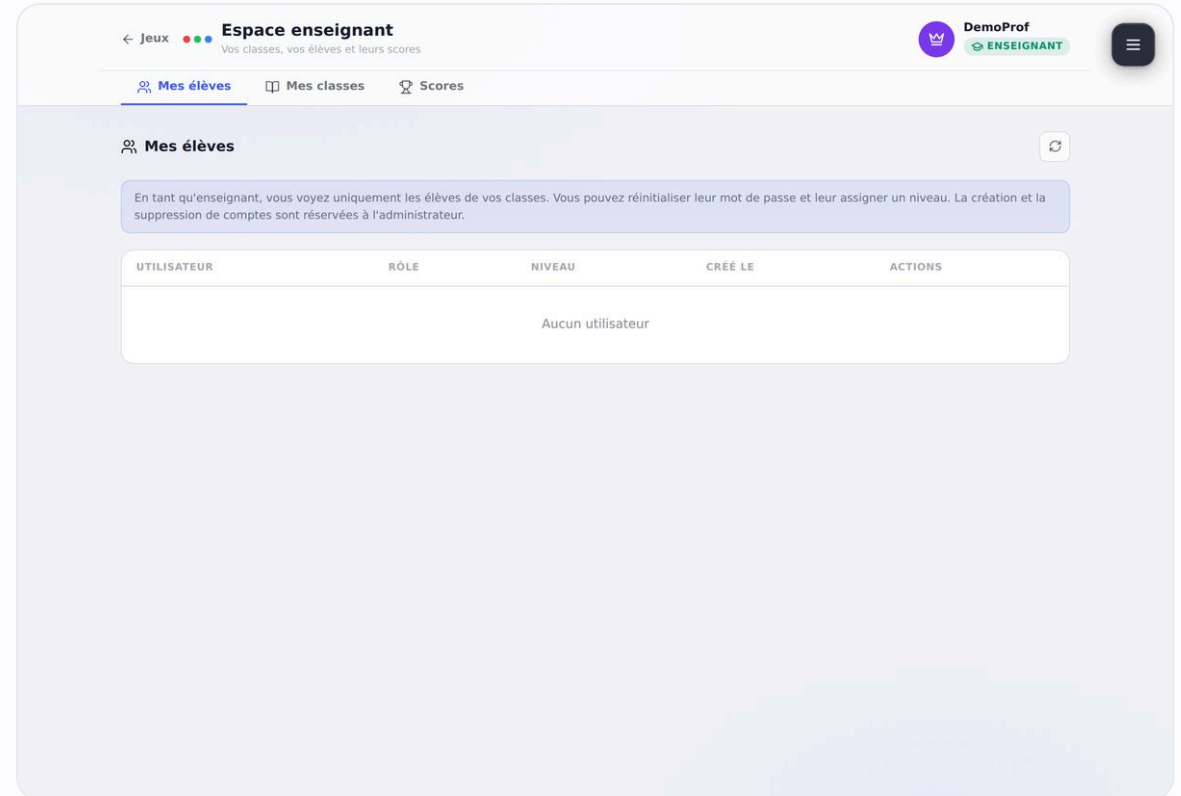




Tableau de bord enseignant

- **Classes** : code de 6 caractères (ex. **CS5VHX** , sans 0/O, 1/I)
+ QR code
- **Suivi des élèves** : niveau et scores par apprenant
- **Gestion des utilisateurs** : rôles, réinitialisation, suppression (admin)
 - Export **CSV** (Blob, BOM UTF-8) · scores en JOIN scores × users





Création de compte et adhésion à une classe

- Inscription guidée en **3 étapes** : pseudo + mot de passe, avatar, classe
- **Rejoindre une classe** en saisissant son **code** (fourni par l'enseignant)
- Création **atomique** (compte + adhésion) puis **connexion automatique**
 - RGPD : un simple **pseudo** suffit, aucune donnée nominative

The screenshot shows the 'ColorRoomGames' web application. At the top, the header includes the logo, the text 'ColorRoomGames', and 'Projet BTS CIEL · ENTPE/LTDS · Session 2026'. A hamburger menu icon is in the top right. The main content area has a light blue background with a large, faint 'ColorRoom' logo. In the center, a white card titled 'Créer votre compte' contains the following elements:

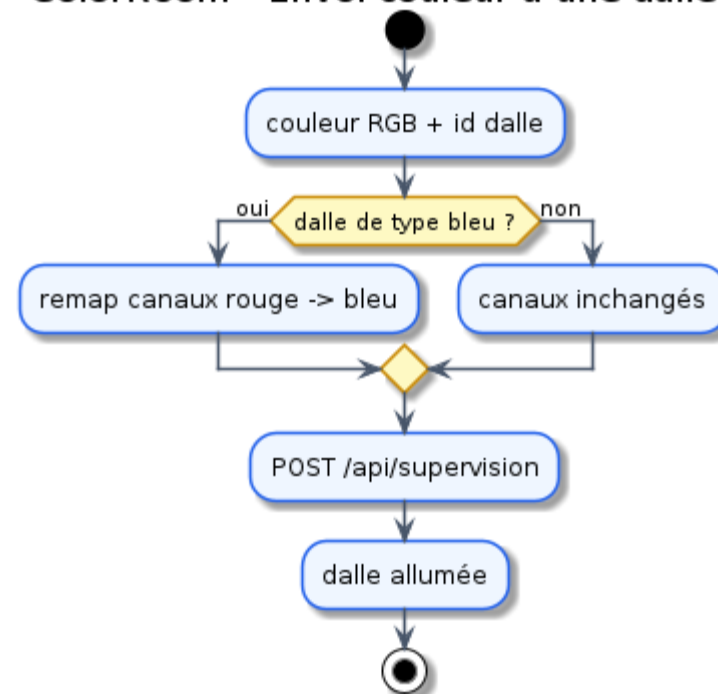
- A progress bar with three steps: 1 (active), 2, and 3 (Compte).
- The title 'Créer votre compte' and a subtitle 'Un pseudo unique et un mot de passe sécurisé.'
- A section labeled 'PSEUDO' with a text input field containing the placeholder 'Choisissez un pseudo...'.
- A section labeled 'MOT DE PASSE' with a text input field containing the placeholder 'Min. 4 caractères' and an eye icon for toggling visibility.
- A section labeled 'CONFIRMER' with a text input field containing the placeholder 'Répéter le mot de passe'.
- A 'Continuer →' button at the bottom.



Remappage des canaux LED

- Entrée : une couleur **RGB** demandée par le jeu
- Conversion vers les **32 canaux** de la plaque
- Application des **profils** (longueur d'onde réelle)
- Envoi de la trame au matériel (proxy supervision)

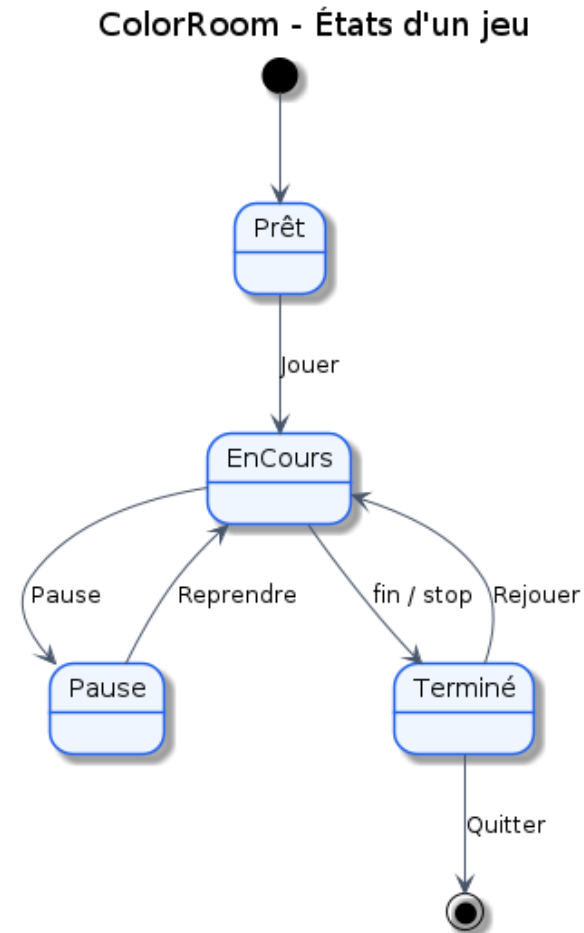
ColorRoom - Envoi couleur à une dalle





Cycle de vie d'une partie

- États : **Prête** → **En cours** → **Terminée**
- Transitions : démarrer, jouer un coup, fin de partie
- Calcul et enregistrement du **score** en fin de partie
- Réinitialisation pour **rejouer**





Puissance 4 et son IA minimax

- Grille 6 × 7 ; 2 joueurs ou IA **hors-ligne**
- **minimax** + **élagage alpha-bêta** : on coupe les branches inutiles (**alpha >= beta**)
- Feuilles évaluées par **evaluateBoard** : fenêtres de 4, **défense > attaque** + poids central
- Coups triés **centre d'abord** (**orderColumns**) pour élaguer plus tôt
- **5 niveaux** réglés par **depth** et **noise** ; victoire = **WIN_SCORE**

</> **app/app/_components/GamePuissance4.tsx**

github.com/NewGenesisAgency/color_room/blob/main/app/app/_components/GamePuissance4.tsx#L174-L199

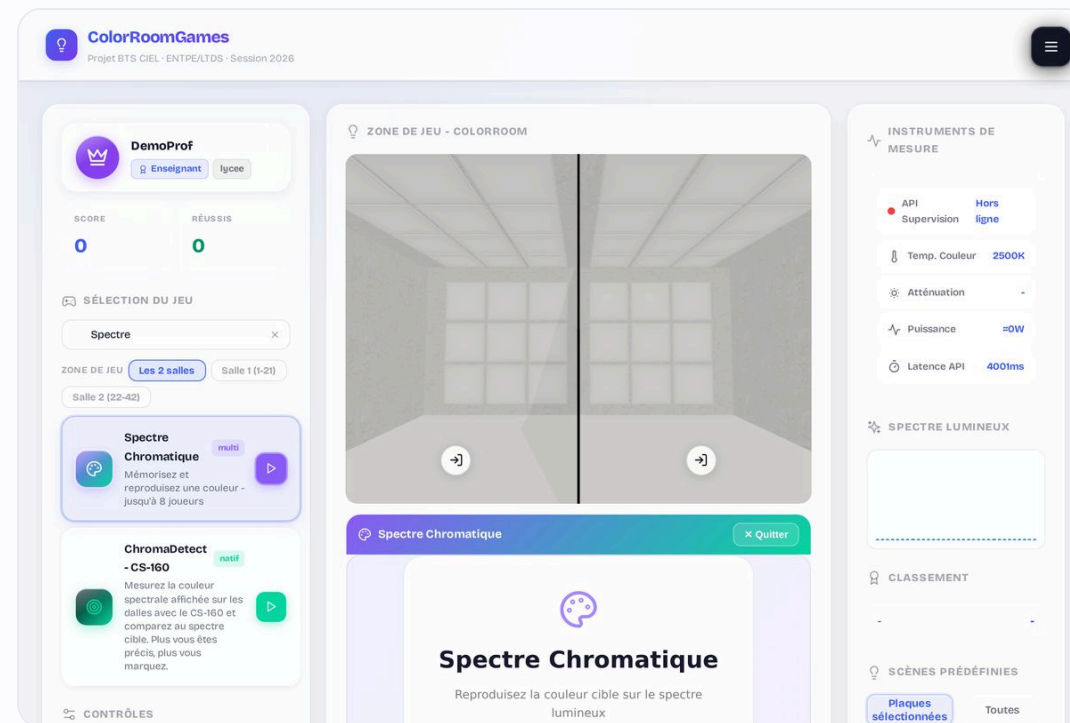
app/app/_components/GamePuissance4.tsx

```
// minimax avec élagage alpha-bêta (depth, noise → difficulté)
function minimax(grid, depth, alpha, beta, maximizing) {
  if (depth === 0) return evaluateBoard(grid); // heuristique
  if (maximizing) { // tour de l'IA
    let value = -Infinity;
    for (const c of orderColumns(valid)) { // centre d'abord
      const d = dropAt(grid, c, AI);
      const v = winsAt(d.grid, d.row, c, AI)
        ? WIN_SCORE + depth // gain immédiat
        : minimax(d.grid, depth-1, alpha, beta, false);
      value = Math.max(value, v);
      alpha = Math.max(alpha, value);
      if (alpha >= beta) break; // coupure bêta
    }
    return value;
  }
  // ... cas minimisant (joueur) : coupure alpha symétrique
}
```



Les jeux multijoueur

- État persisté **en base**, récupéré par **polling** de `/state`
- **Spectre Chromatique** : jusqu'à 8 joueurs (score = distance)
- **Morpion en réseau** : 2 joueurs, détection de victoire
- **Heartbeat** de présence · jetons **UUID** · hôte = siège 1

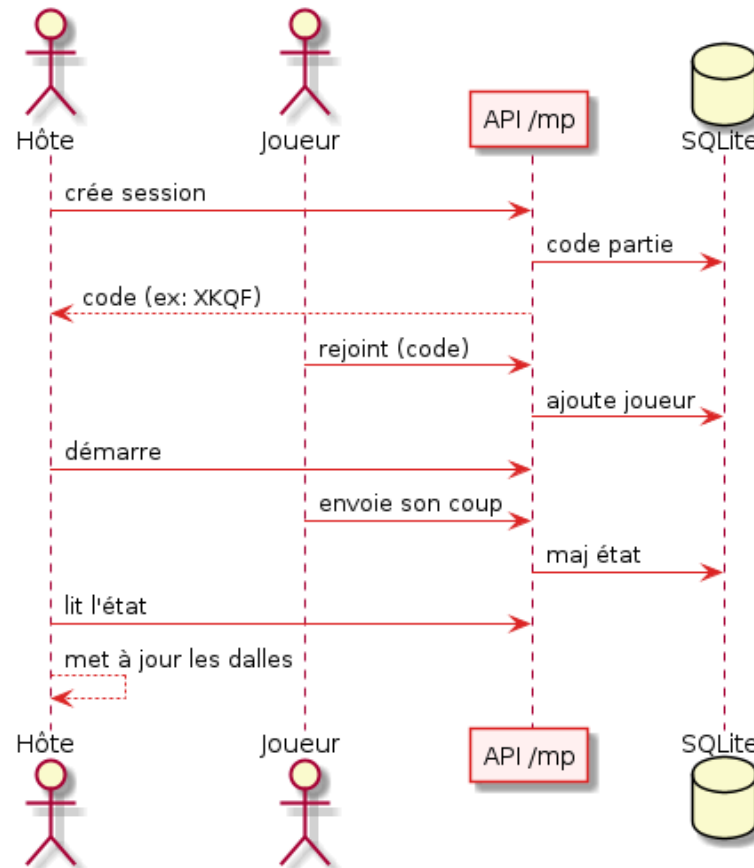




État partagé et interrogation périodique

- L'hôte crée une session et obtient un **code**
- Les invités rejoignent en saisissant ce code
- L'état est **persisté en base** (state_json)
- Chaque client interroge **/state** en **polling** → écrans synchronisés

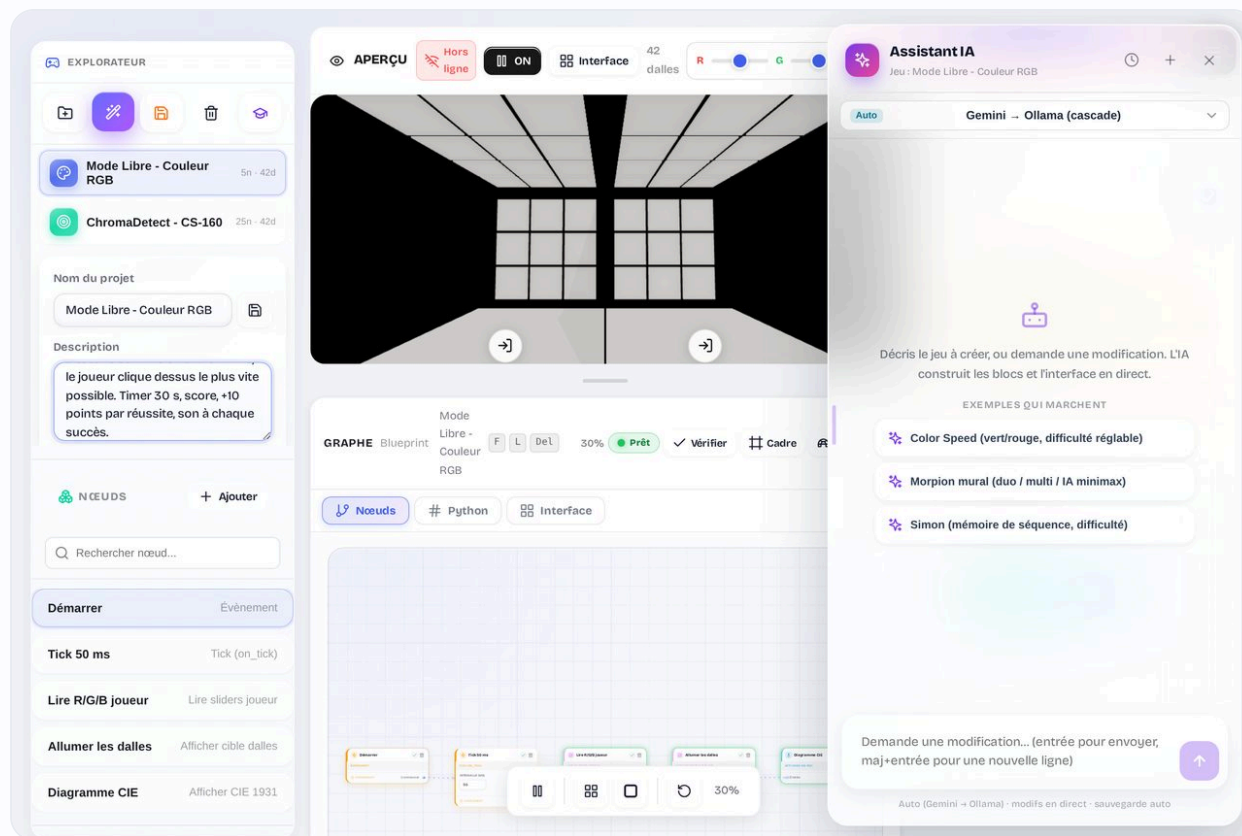
ColorRoom - Session multijoueur





Créer un jeu sans coder

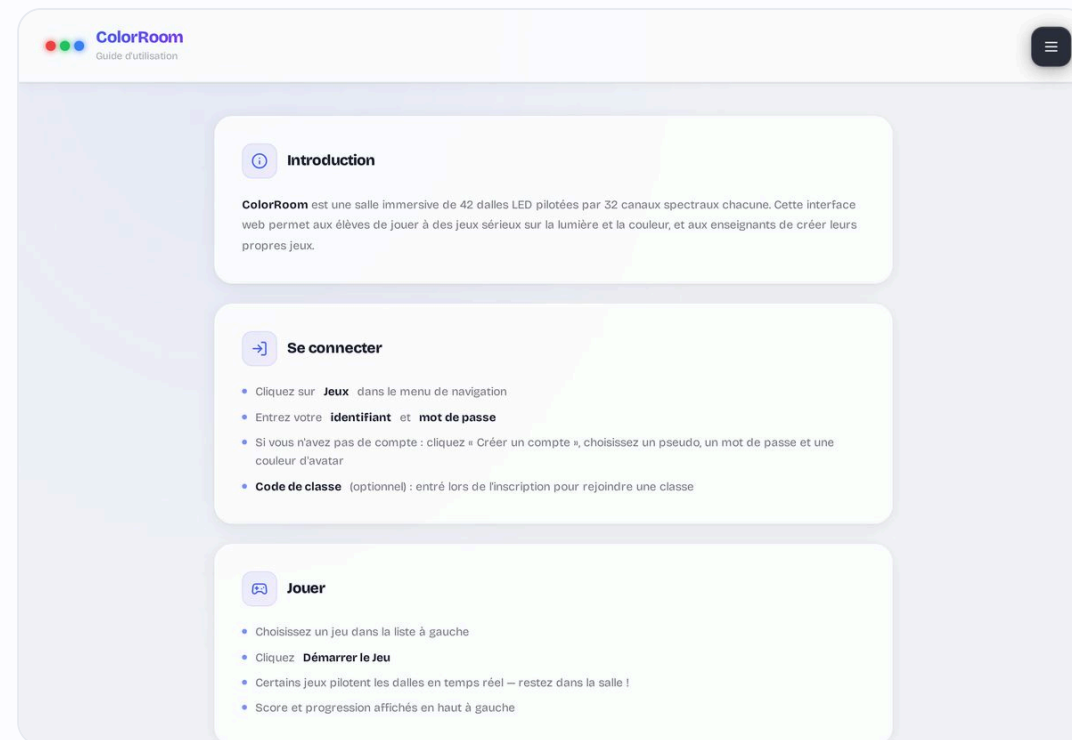
- Éditeur **no-code** (partie E3) : graphe de **nœuds** + aperçu 3D live
- Mon exploration : « **Créer avec l'IA** » génère un jeu complet depuis une phrase
- Route </api/ai/generate-game> → **Ollama** (local) ou Gemini (cloud, optionnel)
 - Garde-fous : reste **éducatif** ; intègre les nœuds CS-160 et dalles





Documentation et robustesse

- Page **/aide** embarquée (hors-ligne) : comment jouer, mesurer, lire le diagramme CIE
- **Documentation** rédigée par moi : guide technique, **15 diagrammes UML**, README d'installation
- Robustesse : **migrations idempotentes**, **transactions ACID**, connexion SQLite en **singleton**
 - Vérif **types (tsc)** + build prod à chaque étape · exports CSV UTF-8 + BOM
 - tests API (E4)





Difficultés rencontrées et solutions

CORS & pare-feu bloquant l'API Supervision



En-têtes CORS + ouverture du port Windows + portproxy

Latence des plaques (32 canaux)

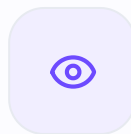


Envoi parallèle (Promise.all) + timeout (AbortController)

Couleur écran différente des dalles



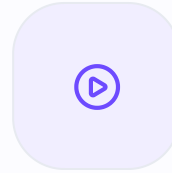
Rendu unifié + profils calés sur les longueurs d'onde



PLACE À LA

Démonstration

Connexion · catalogue et un jeu sur les dalles · Puissance 4 contre l'IA · un jeu multijoueur · mesure CS-160 et diagramme CIE · tableau de bord enseignant



DÉMONSTRATION

Vidéo du projet

Présentation filmée de la ColorRoom en fonctionnement · ≈ 2 min

 [video_demo.mov](#)



POUR LES QUESTIONS DU JURY

Annexes

Code détaillé (variables transactionnelle / atomique / volatile, sécurité) et architecture des dossiers.



Variable transactionnelle (ACID)

- (ACID = Atomicité, Cohérence, Isolation, Durabilité : les garanties d'une transaction)
- `db.transaction()` encapsule un **BEGIN / COMMIT / ROLLBACK**
- L'inscription = INSERT utilisateur + jonction de classe, en **une seule unité**
- **Atomicité** : exception → **ROLLBACK** total, jamais de compte « à moitié créé »
- Requêtes **préparées** (`prepare`) = anti-injection SQL

</> `app/app/api/auth/register/route.ts`

github.com/NewGenesisAgency/color_room/blob/main/app/app/api/auth/register/route.ts#L47-L62

```
app/app/api/auth/register/route.ts

// Variable transactionnelle : tout-ou-rien (ACID).
const insertAll = db.transaction() => {
  db.prepare("INSERT INTO crg_users (id, name,
    user_type, password_hash, ...) VALUES (?,...)")
    .run(id, name, 'apprenant', hash, ...);

  if (classCode?.trim()) { // jonction optionnelle
    const cls = db.prepare("SELECT id FROM
      crg_classes WHERE code = ?").get(code);
    if (cls) db.prepare("INSERT OR IGNORE INTO
      crg_class_members ...").run(rid, cls.id, id);
  }
};

insertAll(); // BEGIN ... COMMIT (ROLLBACK si throw)
```



Variable atomique · sémaphore matériel

- `hwInFlight` = **compteur atomique** des accès au matériel en cours
- **Atomique** de fait : la boucle d'événements de Node est **mono-thread** (pas d'accès simultané réel)
- Au-delà de **2 slots**, les requêtes attendent dans une **file** (Promises)
- `supervision.exe` est **quasi-série** : on borne pour ne pas le saturer

</> [app/app/api/supervision/batch/route.ts](#)
github.com/NewGenesisAgency/color_room/blob/main/app/app/api/supervision/batch/route.ts#L39-L80

```
app/app/api/supervision/batch/route.ts

const HW_CONCURRENCY = 2; // quasi-série
let hwInFlight = 0; // variable atomique
const hwWaiters: Waiter[] = []; // file d'attente

async function acquireHwSlot() {
  if (hwInFlight < HW_CONCURRENCY) {
    hwInFlight++; return true; // slot libre
  }
  return new Promise(r => hwWaiters.push({ resolve: r }));
}

function releaseHwSlot() {
  hwInFlight--; // libère un slot
  drainWaiters(); // réveille le suivant
}
```



État temporaire en mémoire (useRef)

- Une variable **volatile** vit en **mémoire vive** le temps de la partie : **perdue au rechargement**
- **useRef** : valeur mutable **sans re-rendu** (rapide : état de jeu haute fréquence)
- **useState** : volatile **réactif** (re-dessine l'écran quand ça change)
- On ne **persiste** (SQLite) que le **résultat final** : le score

</> [app/app/_components/GameColorSpeed.tsx](#)

github.com/NewGenesisAgency/color_room/blob/main/app/app/_components/GameColorSpeed.tsx#L160-L181

```
app/app/_components/GameColorSpeed.tsx

// Volatile RÉACTIF : re-affiche l'écran quand ça change
const [score, setScore] = useState(0);
const [timeLeft, setTimeLeft] = useState(cfg.duration);

// Volatile PUR : en mémoire, NE déclenche PAS de re-rendu (rapide)
const comboRef = useRef(0); // combo en cours
const lightRef = useRef(0); // dalle allumée
const tileStartRef = useRef(0); // instant d'allumage

comboRef.current++; // mutation directe, sans re-render
// ... en fin de partie SEULEMENT, on persiste le score en base.
```



Hachage des mots de passe (PBKDF2)

- Dérivation lente **PBKDF2-HMAC-SHA512**, 100 000 itérations
- **Sel aléatoire** de 16 octets par compte (anti rainbow-tables)
- Vérification : on **recalcule** avec le même sel et on compare

</> [app/lib/auth.ts](#)

github.com/NewGenesisAgency/color_room/blob/main/app/lib/auth.ts#L19-L36

```
app/lib/auth.ts

export function hashPassword(password) {
  const salt = randomBytes(16).toString('hex');
  const hash = pbkdf2Sync(password, salt, 100_000, 64, 'sha512');
  return salt + ':' + hash.toString('hex'); // format sel:hash
}

export function verifyPassword(pw, stored) {
  const [salt, hash] = stored.split(':');
  const calc = pbkdf2Sync(pw, salt, 100_000, 64, 'sha512');
  return calc.toString('hex') === hash; // recalcule + compare
}
```




Où se trouve chaque partie

arborescence

```
app/
├ app/          # Next.js (App Router)
│ ├── _components/  # UI + jeux (Room3D, P4)
│ ├── api/         # routes serveur
│ │ ├── auth/ supervision/ cs160/
│ │ └── multiplayer/ scores/ games/
│ └── jeux/ editeur/ mesure/ gestion/ # pages
├ lib/          # logique partagée
├ db/           # SQLite + migrations
├ auth.ts       # hachage + sessions
├ multiplayer.ts settings.ts
└ public/ Dockerfile package.json
```

- **app/app/api/** : tout le **back** (mes Route Handlers)
- **app/app/_components/** : l'**interface** et les **jeux**
- **app/lib/** : **base de données, auth, services**
 - Un seul projet = front + back ensemble